

Backend Take-Home Guide

Format: Paid take-home assignment

Expected duration: This is a 4-hour task

Start here

What you are looking at now:

- Branch: main
- Contents: this guide only (no source code on main)

Your coding work lives on another branch in the same repository.

Next step: choose a branch

This PDF on main is the process overview. To read the full task guide and start coding, check out one of the coding branches and open its README.md.

Repository layout (branches):

```
backend/
+-- main                <- you are here
|   +-- HOME_TASK_GUIDE.pdf
|       Process overview only (no source code)
|
+-- scratch              <- checkout to start
|   +-- README.md
|       Full requirements (choose your own stack)
|
+-- boiler-node          <- checkout to start (Node.js + TypeScript)
+-- boiler-go            <- checkout to start (Go)
+-- boiler-rust          <- checkout to start (Rust)
+-- boiler-java          <- checkout to start (Java Spring Boot)
```

Important: If you still only see this PDF in the "backend" folder, you are on main. Switch branches to continue.

1. Purpose

Build a Vesting Service API for a Web3 backend. Focus on correctness of vesting math, transaction/release workflow modeling, API design, and test quality.

2. Process

1. Read this guide
2. Choose one option:
 - scratch - implement in your preferred backend stack
 - boiler-node - Node.js + TypeScript starter
 - boiler-go - Go starter
 - boiler-rust - Rust starter

- boiler-java - Java Spring Boot starter

3. Build and test

4. Submit your work from this repository (or your fork)

```
git checkout scratch
# or
git checkout boiler-node
# or
git checkout boiler-go
# or
git checkout boiler-rust
# or
git checkout boiler-java
```

3. Product summary

Create an API that supports:

- Create vesting schedule
- Compute vested and releasable amount at arbitrary timestamps
- Release currently available amount
- Return schedule and release history
- Validation and error handling for invalid requests

4. What we evaluate

- Vesting math correctness
- API contract quality and input validation
- Code architecture and maintainability
- Test depth and edge-case coverage
- README clarity

5. Guidelines

1. This is a 4-hour task
2. Scratch branch supports Node, Go, Rust, Java, C++, C#, or another stack
3. Boiler branches are language-specific starters
4. Include how to install/run/test

6. Submission

Submit your work from this repository (or your fork), including:

- Branch name used (`scratch` or a `boiler-\*` branch)
- Install / run / test commands
- 5-10 bullet design notes (assumptions, trade-offs, known gaps)
- Time spent

7. Compensation

Paid take-home. Compensation applies whether or not we proceed.